NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *January 12, 2026*

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão
Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel
Associate Professor, Reykjavik University

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: *January 12, 2026*

Abstract

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis LaTeX (novathesis) (LaTeX) template will respect them. Otherwise, the default rule in the novathesis template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more · The last keyword

Sustainable Development Goals (SDG):

Contents

List of Figures

List of Tables

Acronyms

novathesis NOVAthesis LaTeX *(p. i)*

1

Introduction

2

Background

2.1 Software modelling & conceptual modelling

2.1.1 Purpose and role of modelling in software engineering

Software systems are complex socio-technical constructs that must meet a variety of functional and non-functional requirements while evolving under technical, organisational, and human constraints. Within this context, software modelling supports reasoning about software systems by providing abstractions that capture relevant aspects of reality at an appropriate level of detail. Models serve as blueprints for design, facilitate communication among stakeholders, and enable analysis and validation of system properties before implementation [13, 5].

While modelling is frequently introduced through widely adopted notations such as the UML or BPMN, it is important to recognise that modelling should not be reduced to "drawing diagrams". Instead, models should be viewed and treated as structured representations based in a modelling language's semantics and intended to serve a clear purpose for specific stakeholders within a given context. This perspective frames modelling not merely as documentation, but as a means of constructing and manipulating knowledge about a domain or system [5].

2.1.2 Conceptual modelling as a broader discipline

It is relevant to differentiate between software modelling as a practice within software engineering, and conceptual modelling as a broader discipline that studies how to represent, structure, and reason about domains and systems across various fields. Conceptual modelling is not limited to software development alone; it is also applied in organisational analysis, enterprise and service design, data-intensive systems, and other domains where abstraction is needed to manage complexity and facilitate understanding [13]. Mappings of conceptual modelling education also reinforce its interdisciplinary nature, while highlighting that conceptual modelling spans a broad family of modelling subtypes (such as data, process, goal, and enterprise modelling) that often evolve with limited pedagogical

or theoretical integration [13, 3, 2].

This disciplinary perspective is particularly relevant for research on teaching and learning software modelling. If modelling is treated merely as a supplementary topic within software engineering courses, students may not fully appreciate its significance or develop oversimplified views of its purpose and application. The conceptual modelling perspective argues instead that modelling languages resemble knowledge schemas: they define what can be expressed, how meaning is structured, and what kinds of analysis or tooling become possible. Therefore, understanding modelling requires engaging with both notational elements and the underlying semantics, as well as the modelling goals and the intended audience of the produced models [5].

2.1.3 Modelling methods, languages, and artefacts

Several authors conceptualise a modelling method as more than a set of symbols or notations. A modelling method can be understood as comprising a modelling language (including syntax, semantics, and notation), a modelling procedure (the process or steps to create models), and supporting mechanisms and algorithms such as analysis, simulation, or transformation capabilities [4, 5]. This holistic view clarifies why students may be able to reproduce symbols yet still struggle to apply modelling meaningfully: the method's procedural and analytical aspects are often less explicitly taught or emphasised.

From this viewpoint, the value of a modelling language is not only in enabling representation, but also in supporting reasoning. When a language is treated as a structured schema, models become more than static diagrams: they can become queryable, analysable, and transformable artefacts that facilitate deeper engagement with the domain. This opens the door to tool support that can provide feedback on syntactic correctness, highlight semantic issues, help manage complexity, or even automate certain modelling tasks [2, 13].

2.1.4 Implications for the scope of this thesis

This thesis adopts the view that software modelling is an instantiation of the broader discipline of conceptual modelling. It therefore treats modelling as a discipline involving:

1. Conceptual understanding of modelling constructs and their semantics,

2. Procedural competence in constructing and refining models from problem descriptions,

3. Analytical judgement of model quality and fit-for-purpose, and

4. Socio-technical awareness of how modelling practices are shaped by tools, learning contexts, and stakeholder needs.

These aspects resonate with existing work on modelling learning outcomes and competencies, which emphasises not only knowledge of notations but also procedural skills, evaluative abilities and reflective understanding [2, 3, 4, 14].This framing motivates the later background sections on modelling education, grounded theory, and learning outcomes, and provides the conceptual foundation for investigating challenges in learning and teaching modelling from both student and instructor perspectives.

2.2 Modelling education (conceptual, process, goal, UML, MDE)

2.2.1 Overview of conceptual modelling education

Conceptual and software modelling are widely taught across computing and software-related programs, but the way these topics are taught and integrated into curricula remains fragmented. Maslov et al. [13] map the conceptual modelling education literature and identify that educational work is scattered across various modelling sub-communities (e.g., data modelling, process modelling, enterprise modelling, etc.), which often evolve in parallel with limited reuse of frameworks, terminology, and learning theories across subfields [13]. Within this landscape, modelling appears in diverse course contexts (from introductory programming courses to advanced software engineering), and courses emphasise different modelling families and purposes (e.g., data-centric, process-centric, goal-oriented, etc.).

The same studies also highlight that much of the current practice in modelling education is driven by instructor experience and preferences, local institutional constraints, and the availability of tools, rather than by systematically articulated educational frameworks or empirically validated design principles [13]. While there is a broad consensus on the importance of modelling skills for software professionals, there is less agreement on which specific *competencies* learners should acquire, how these competencies should be scaffolded, and how learning should be assessed in a reliable and educationally meaningful way.

2.2.2 Bloom-based perspectives on modelling learning outcomes

In response to the fragmented state of modelling education, several authors have proposed to adapt Bloom's revised taxonomy to conceptual and domain modelling, with the goal of making learning outcomes more explicit and comparable across courses and contexts. At a general level, these works start from the cognitive levels of remembering, understanding, applying, analysing, evaluating, and creating [9], and specialise them to the kinds of activities involved in modelling, such as interpreting models, constructing models from textual descriptions, critiquing model quality, or designing new modelling methods.

Bogdanova and Snoek analyse learning outcomes and assessment tasks for domain modelling through the lens of Bloom's revised taxonomy, identifying that existing tasks tend to concentrate on understanding, analysing, and creating conceptual knowledge while

giving less attention to evaluation, procedural knowledge, or metacognition [3]. Building on this analysis, CaMeLOT provides a three-dimensional framework that combines Bloom's cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and specific content areas of conceptual data modelling (e.g., classes, relationships, methods, quality, and tool support) [2]. This framework is instantiated with detailed example learning outcomes and can be used to design or audit courses.

Bork extends this Bloom-based perspective to both conceptual modelling and metamodeling, mapping knowledge and cognitive dimensions to all components of a modelling method (language, semantics, notation, procedure, mechanisms and algorithms) [4]. An evaluation of a "Smart City" teaching case using this framework reveals that even rich, practice-oriented material may still underemphasise certain cognitive levels (e.g., evaluation) or knowledge types (e.g., metacognitive knowledge). These works collectively suggest that Bloom's taxonomy can provide a useful lens for reflecting on and improving modelling education, by making learning objectives more explicit and balanced across different dimensions.

2.2.3 Process and goal modelling education

Beyond data and structural modelling, there is growing interest in how students learn process and goal-oriented modelling. For business process modelling with BPMN, Maslov outlines a research agenda for developing empirically validated process-moddelling education that integrates insights from conceptual modelling education, process model quality research, and learning sciences [18]. This agenda highlights the need for systematic mapping of existing work, empirical studies of novice modellers' behaviour and typical errors, and the design and experimental evaluation of BPMN-specific teaching strategies and feedback mechanisms. This reflects a broader recognition that teaching process modelling effectively requires more than just introducing notation; it requires attention to cognitive demands, quality criteria, and the kinds of feedback needed to help learners improve.

Goal and actor-oriented modelling introduces additional layers of complexity, as it requires learners to reason about stakeholder intentions, rationales, and dependencies. The iStar learning study explicitly investigates the challenges students face when learning iStar, using a combination of Bloom-based task framing and grounded theory analysis of tutorials, modelling sessions, and interviews [19]. The authors report difficulties at multiple cognitive levels, including understanding the semantics of iStar constructs, applying the notation to real requirements, and analysing models to assess completeness and appropriateness. They derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, visualisation, and semi-automation modelling from text) that could help address these challenges [19].

These strands of work on BPMN and iStar education illustrate that different modelling families pose distinct learning challenges and may require specialised pedagogical

approaches. They also reinforce the broader themes identified in conceptual modelling education: the need for scaffolding from text to model, support for managing complexity, and explicit attention to how learners evaluate and refine their models.

2.2.4 Modelling, tools, and model-driven engineering in education

2.2.5 Competence-oriented and human-centred perspectives

2.2.6 Implications for this thesis

2.3 Grounded Theory

2.3.1 Overview and key characteristics

2.3.2 Grounded Theory in software engineering

2.3.3 Socio-Technical Grounded Theory

2.3.4 Implications for this thesis

2.4 Bloom's Taxonomy and learning outcomes

2.5 Human factors in modelling and SE education

3

Related Work

3.1 Empirical studies on learning/teaching specific modelling frameworks (iStar, UML, data models)

3.2 Broader SE education work on experience, competencies, experiential learning

3.3 Existing theories/frameworks (CaMeLOT, Domain Modelling in Bloom, etc.)

4

Methodology

4.1 Research design and rationale

4.2 Research questions

4.3 Study context and participants

4.4 Data collection

4.4.1 Instruments and pilot

4.4.2 Interviews

4.4.3 Surveys

4.4.4 Additional artefacts

4.5 Data analysis

4.5.1 Grounded Theory variant and socio-technical stance

´

4.5.2 Coding process

4.5.3 Theoretical sampling and saturation

4.5.4 Bloom's taxonomy as a sensitising concept

4.6 Rigour and trustworthiness

4.7 Ethics and data management

4.8 Methodological limitations

Bibliography

[1] S. Adolph, W. Hall, and P. Kruchten. "Using Grounded Theory to Study the Experience of Software Development". In: *IEEE Software* 28.4 (2011), pp. 24–27.

[2] D. Bogdanova and M. Snoeck. "CaMeLOT: An Educational Framework for Conceptual Data Modelling". In: *TBD* (2020) (cit. on pp. 3–5).

[3] D. Bogdanova and M. Snoeck. "Domain Modelling in Bloom: Deciphering How We Teach It". In: *TBD* (2020) (cit. on pp. 3–5).

[4] D. Bork. "A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom's Revised Taxonomy". In: *TBD*. 2020 (cit. on pp. 3–5).

[5] R. A. Buchmann, D. Karagiannis, and H.-G. Fill. "Conceptual Modelling in Education: A Position Paper". In: *TBD*. 2020 (cit. on pp. 2, 3).

[6] R. Hoda, J. Noble, and S. Marshall. "Socio-Technical Grounded Theory for Software Engineering". In: *IEEE Transactions on Software Engineering* 44.12 (2018), pp. 1208–1226.

[7] R. Holmes, R. J. Walker, and G. C. Murphy. "Dimensions of Experientialism for Software Engineering Education". In: *TBD* (2014).

[8] B. Kitchenham and S. L. Pfleeger. "Teaching Survey Research in Software Engineering". In: *ACM SIGSOFT Software Engineering Notes* 33.6 (2008), pp. 1–10.

[9] D. R. Krathwohl. "A Revision of Bloom's Taxonomy: An Overview". In: *Theory Into Practice* 41.4 (2002), pp. 212–218 (cit. on p. 4).

[10] G. Liebel et al. "Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML". In: *TBD* (2014).

[11] J. Linaker, P. Runeson, and B. Regnell. "Challenges in Survey Research". In: *Empirical Software Engineering* 20.4 (2015), pp. 924–962.

[12] J. M. Lourenço. *The NOVAthesis LaTeX Template User's Manual*. NOVA University Lisbon. 2021. URL: https://github.com/joaomlourenco/novathesis/raw/main/template.pdf.

[13] A. Maslov, S. Poelmans, and N. Rosenthal. “Conceptual Modeling Education”. In: *TBD* (2021) (cit. on pp. 2–4).

[14] Y. Sedelmaier and D. Landes. “A Research Agenda for Identifying and Developing Required Competencies in Software Engineering”. In: *TBD* (2014) (cit. on p. 4).

[15] K.-J. Stol, P. Ralph, and B. Fitzgerald. “Grounded Theory in Software Engineering Research: A Critical Review and Guidelines”. In: *IEEE Transactions on Software Engineering* 42.12 (2016), pp. 1201–1222.

[16] TBD. “Domain Diversity, Motivation, and Inclusion in Software Modelling Education”. In: *TBD*. 2020.

[17] TBD. “Human Factors in Model-Driven Engineering: Future Research Goals and Initiatives for MDE”. In: *TBD*. 2020.

[18] TBD. “Towards Empirically Validated Process Modelling Education Using a BPMN Formalism”. In: *TBD*. 2020 (cit. on p. 5).

[19] TBD. “Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners”. In: *TBD*. 2017 (cit. on p. 5).